DIGITAL VLSI DESIGN

Assignment 1

4-bit × 4-bit Unsigned Array Multiplier from RTL to GDSII

SkyWater SKY130 (sky130_fd_sc_hd) · open-source tools

Team member	Roll number
Jay Daulatkar	2023102044
Parth Tokekar	2023102041

Repository: https://github.com/jaydaulatkar/DVD_A1

Tools (IIC-OSIC-TOOLS container `hpretl/iic-osic-tools`): Icarus Verilog 14.0, GTKWave 3.3.116, Yosys 0.70, OpenSTA 3.1.0, LibreLane (OpenROAD, Magic, KLayout 0.30.12, Netgen). Parts B and C use the typical corner `tt_025C_1v80`.

The exact command of every step is listed in `README.md`.

Contents

1 Part A — RTL design and verification 3

Simulation results . . . 3

Hand-worked example (task 5) . . . 3

Waveform (A = 9, B = 7) . . . 4

Block diagram . . . 5

2 Part B — Logic synthesis with Yosys 6

3 Part C — Static timing analysis with OpenSTA 7

Cells on the critical path (from mult_array_timing.rpt, T = 2.79 ns) . . . 7

4 Part D — Physical design with LibreLane (RTL → GDSII) 9

Layout screenshots (KLayout, final GDS) . . . 11

5 Additional verification 12

6 Answers to the questions (Section 10 of the assignment) 13

1. Logic cells after synthesis. . . . 13

2. Critical path. . . . 13

3. Flip-flop share of the area. . . . 13

4. Why post-route Fmax is lower; added cells. . . . 13

5. DRC and LVS. . . . 14

6. Scaling to 8 bits. . . . 14

Results at a glance

Part	Key results
A — RTL design and verification	PASS: 256 vectors checked, 0 errors; error test (sum = a ^ b): FAIL: 256 vectors checked, 95 errors; 9 × 7 = 63 seen 2 clock cycles after the inputs
B — Logic synthesis	61 cells (16 × dfrtp_1), 756.976 µm², flip-flops 52.89 % of the area; gate-level simulation PASS
C — Static timing analysis	WNS 7.21 ns at 10 ns → T_{min} = 2.79 ns, F_{max} = 358.42 MHz; critical path a_r[1] → p[6] through 8 full-adder stages
D — Physical design	CLOCK_PERIOD 8.0 ns after 10 runs; setup slack +0.396 ns; post-route F_{max} 131.51 MHz; 1883 instances; 0 DRC / 0 LVS errors

1 Part A — RTL design and verification

Design. `full_adder` is written with two `assign` statements (sum = a⊕b⊕cin, cout = ab + a·cin + b·cin). `mult_array` registers a and b into `a_r`/`b_r` on every rising edge (asynchronous active-low reset `rst_n`), forms the 16 partial-product bits $pp_i[j] = a_r[j] \cdot b_r[i]$, adds them with 12 `full_adder` instances in three 4-bit ripple-carry rows (generate loops `row1`/`row2`/`row3`) wired exactly as in the connection table of Section 2, and registers the 8-bit result into p. No * operator is used. This gives 16 flip-flops (4 + 4 + 8) and a latency of 2 clock cycles.

Testbench. `tb_mult` generates a 10 ns clock, holds `rst_n` low for the first 2 cycles (and checks that the reset has cleared p), applies all 256 combinations on the falling edge, waits for the two rising edges of the latency, compares p with `a*b` half a cycle after edge 2, prints a line per mismatch and exactly one summary line, and dumps `mult.vcd`.

Simulation results

```
$ iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v && vvp sim
PASS: 256 vectors checked, 0 errors

# task 6: sum changed to  assign sum  = a ^ b;  (then undone)
MISMATCH at 1060 ns: a=3 b=3 expected=9 got=5
MISMATCH at 1120 ns: a=3 b=6 expected=18 got=10
MISMATCH at 1140 ns: a=3 b=7 expected=21 got=9
...
FAIL: 256 vectors checked, 95 errors
```

The error-injection test is caught: removing `cin` from the sum makes 95 of the 256 products wrong. The change was undone afterwards and the testbench passes again.

Hand-worked example (task 5)

Roll number used: **2023102044 (Jay Daulatkar)**. Last digit 4, so **A = 4 + 5 = 9 = 1001**; **B = 7 = 0111** (A ≠ 7, so B = 7).

```
            1 0 0 1      A = 9
          x 0 1 1 1      B = 7
          ---------------
            1 0 0 1      pp0 = A AND b0 (b0 = 1)
          1 0 0 1        pp1 = A AND b1 (b1 = 1), shifted 1
        1 0 0 1          pp2 = A AND b2 (b2 = 1), shifted 2
      0 0 0 0            pp3 = A AND b3 (b3 = 0), shifted 3
          ---------------
      0 0 1 1 1 1 1 1    P = 63
```

```
Row by row through the array (bits written MSB..LSB):
  row 0: pp0 = 1001 -> p[0] = 1,  r0 = {0,pp0[3:1]} = 0100
  row 1: pp1 1001 + r0 0100 = s1 1101, c1[4] = 0 -> p[1] = 1,  r1 = {c1[4],s1[3:1]} = 0110
  row 2: pp2 1001 + r1 0110 = s2 1111, c2[4] = 0 -> p[2] = 1,  r2 = {c2[4],s2[3:1]} = 0111
  row 3: pp3 0000 + r2 0111 = s3 0111, c3[4] = 0 -> p[3] = 1,  p[7:4] = {c3[4],s3[3:1]} = 0011
  P = 0011 1111 = 63 = 9 x 7
```

Waveform (A = 9, B = 7)

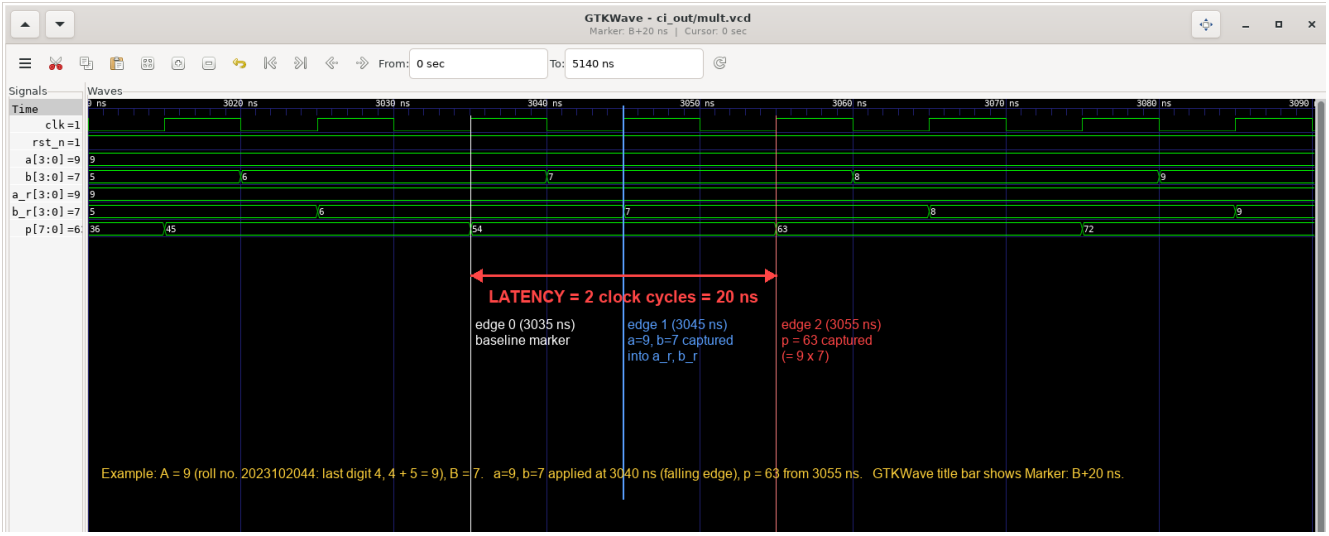


Figure 1. *GTKWave screenshot of mult.vcd (clk, rst_n, a, b, internal a_r, b_r, and p). The inputs a = 9, b = 7 are applied at 3040 ns, captured into a_r/b_r at clock edge 1 (3045 ns), and the product p = 63 is captured at edge 2 (3055 ns). Baseline marker at edge 0 (3035 ns), marker at edge 2: the title bar shows Marker: B+20 ns = 2 clock cycles.*

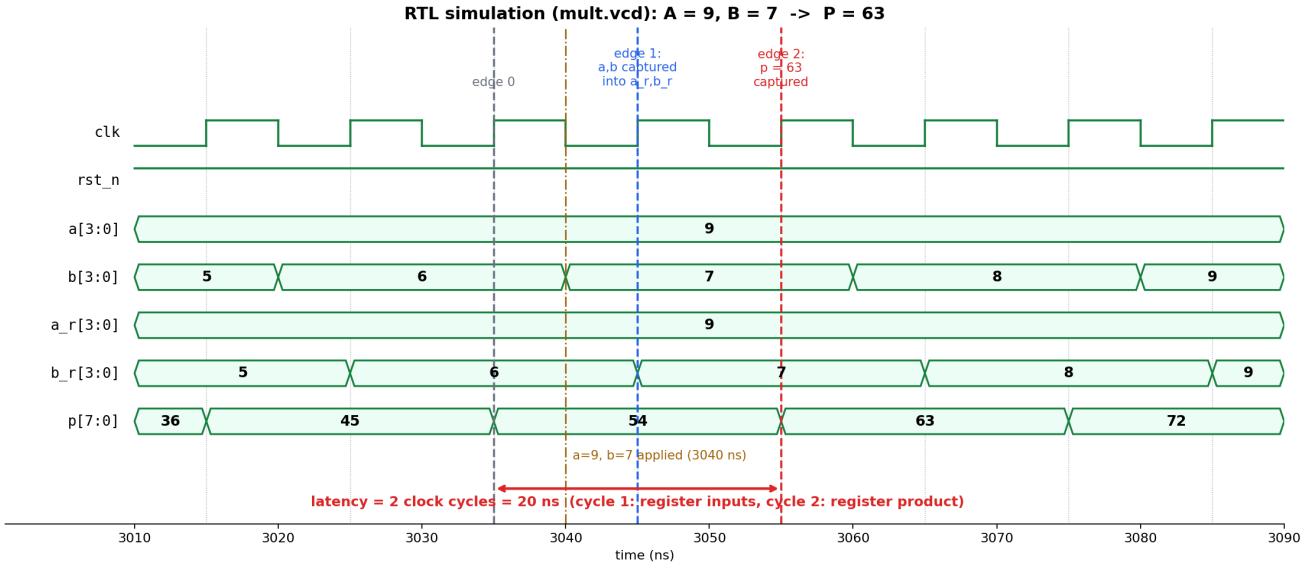
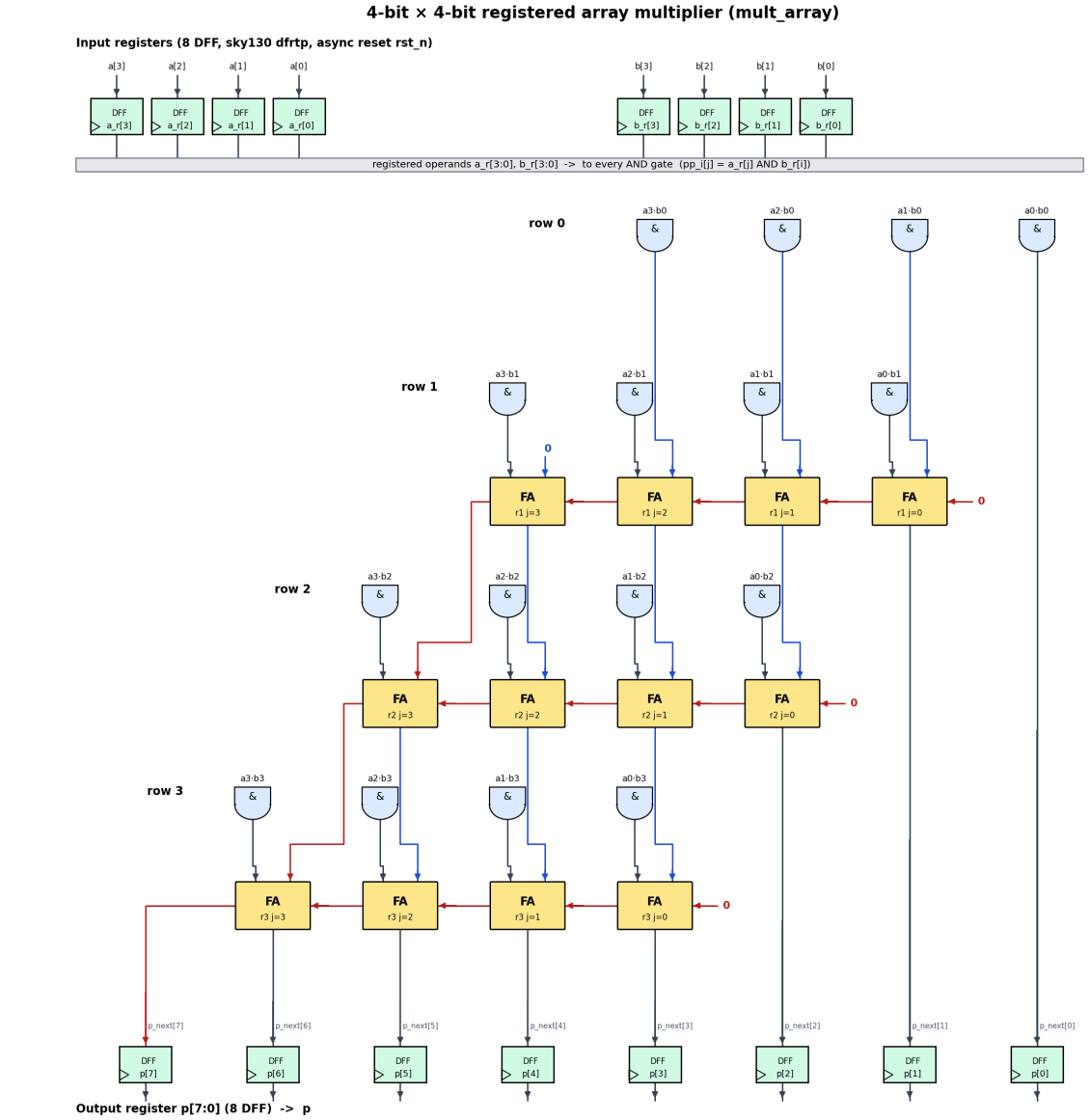


Figure 2. *The same window plotted from mult.vcd with the 2-cycle latency marked.*

Block diagram



16 AND gates (pp_i[j] = a_r[j]·b_r[i]), 12 full adders in rows 1–3 (4-bit ripple-carry each), 16 flip-flops (4 a_r + 4 b_r + 8 p). clk -> all DFFs (rising edge), rst_n -> async clear.
Blue = bits passed down to the next row (r_i), red = carry chain / row carry-out, black = partial products and finished product bits. Latency 2 cycles, throughput 1 result/cycle.

Figure 3. *mult_array: input registers a_r/b_r (8 DFF), 16 AND gates, 12 full adders in rows 1–3, output register p (8 DFF).*

2 Part B — Logic synthesis with Yosys

`synth/synth.tcl` runs the seven required steps (read_verilog → synth -top mult_array -flatten → dfflibmap → abc → opt_clean -purge → tee -o synth/mult_array_stat.rpt stat -liberty → write_verilog -noattr), with $lib = `sky130_fd_sc_hd__tt_025C_1v80.lib`.

Table 1. Part B results.

Metric	Value
Total number of cells	61
Number of flip-flops	16 (sky130_fd_sc_hd__dfrtp_1)
Total cell area (μm²)	756.976
Flip-flop area (μm²)	16 × 25.024 = 400.384
Flip-flop area (% of total)	52.89 %
The 3 most-used cell types and their counts	dfrtp_1 × 16, nand2_1 × 7, and2_0 × 4 and maj3_1 × 4 (tied for third)
Netlist simulation result (PASS/FAIL line)	PASS: 256 vectors checked, 0 errors

Table 2. Synthesized cells (synth/mult_array_stat.rpt).

Cell	Count	Area (μm²)
dfrtp_1	16	400.384
nand2_1	7	26.275
and2_0	4	25.024
maj3_1	4	40.038
a21oi_1	3	15.014
xnor2_1	3	26.275
xor3_1	3	71.318
a22oi_1	2	15.014
clkinv_1	2	7.507
lpflow_isobufsrc_1	2	12.512
nand3_1	2	10.010
nand4_1	2	12.512
nor2_1	2	7.507
xor2_1	2	17.517
a21boi_0	1	7.507
and3_1	1	6.256
and4_1	1	8.758
o2111ai_1	1	8.758
o21bai_1	1	7.507
o311ai_0	1	8.758
xnor3_1	1	22.522

Gate-level simulation (exact command of Part B, -DFUNCTIONAL -DUNIT_DELAY=#1 with primitives.v and sky130_fd_sc_hd.v): **PASS: 256 vectors checked, 0 errors**, so synthesis did not change the behaviour.

3 Part C — Static timing analysis with OpenSTA

`sta/sta.tcl` performs the seven required steps (read_liberty, read_verilog + link_design, create_clock, set_input_delay 1.0, set_output_delay 1.0, report_worst_slack -max, report_checks -path_delay max -digits 3 > sta/mult_array_timing.rpt). The period T is read from the environment (default 10 ns).

```
$ sta sta/sta.tcl               -> worst slack max 7.21      (T = 10 ns)
  Tmin = T - WNS = 10 - 7.21 = 2.79 ns,   Fmax = 1000 / 2.79 = 358.42 MHz
$ T=2.79 sta sta/sta.tcl        -> worst slack max 0.00      (report: slack 0.003 ns, MET)
```

Table 3. Part C results.

Metric	Value
Worst slack at T = 10 ns (ns)	7.21
T_{min} (ns)	2.79
F_{max} (MHz)	358.42
Slack at T = T_{min} (ns)	0.003 (≈ 0, MET)
Critical path start point	_91_/CLK — the input-register flip-flop a_r[1] (dfrtp_1)
Critical path end point	_88_/D — the output-register flip-flop p[6] (dfrtp_1)
Number of cells on the critical path	8 combinational cells (10 instances including the launch flip-flop _91_ and the capture flip-flop _88_)
Latency (ns) = 2 × T_{min}	5.58
Throughput (million results per second)	358.42

Cells on the critical path (from mult_array_timing.rpt, T = 2.79 ns)

#	Pin	Cell	Incr. (ns)	Arrival (ns)	Net equals (RTL)	Full adder of the array
0	_91_/CLK	dfrtp_1	0.000	0.000	clock edge at launch FF	—
1	_91_/Q	dfrtp_1	0.398	0.398	a_r[1]	launch register a_r[1]
2	_37_/Y	clkinv_1	0.073	0.471	NOT a_r[1]	FA(1,0) carry c1[1] = pp1[0]·pp0[1],
3	_42_/Y	o311ai_0	0.356	0.827	(merged logic)	FA(1,1) sum s1[1] and
4	_45_/Y	a21oi_1	0.153	0.980	c2[1]	FA(2,0) carry c2[1] = pp2[0]·s1[1] (3 cells)
5	_51_/X	maj3_1	0.402	1.382	c2[2]	FA(2,1) carry
6	_58_/X	xnor3_1	0.370	1.752	NOT s2[2]	FA(2,2) sum (= r2[1])
7	_61_/X	maj3_1	0.368	2.119	NOT c3[2]	FA(3,1) carry
8	_63_/X	maj3_1	0.354	2.474	NOT c3[3]	FA(3,2) carry
9	_64_/X	xor3_1	0.178	2.652	s3[3] = p_next[6]	FA(3,3) sum
10	_88_/D	dfrtp_1	0.000	2.652	p_next[6]	D pin of capture FF p[6] (endpoint)

Data arrival time 2.652 ns, setup time 0.135 ns. To relate the cells to the array, the synthesized netlist was simulated for all 256 inputs with the official SKY130 functional cell models and every net on the path was compared with the RTL signals (column 'Net equals'). In RTL terms the path is a_r[1] → pp0[1] → FA(1,0) carry → FA(1,1) sum → FA(2,0) carry → FA(2,1) carry → FA(2,2) sum → FA(3,1) carry → FA(3,2) carry → FA(3,3) sum = p[6]: it passes through **8 full adders, exactly the 2N = 8 estimate** of Section 2 (down the rows and then along the carry chain of row 3). Synthesis implements these 8 full-adder stages with only 8 cells: FA(1,0) and FA(2,0) have a constant-0 carry input (they are half adders) and are merged with the AND gates and FA(1,1) into the 3 cells clkinv / o311ai / a21oi, while each of the remaining 5 stages is a single complex cell (maj3 for a carry, xor3 /

xnor3 for a sum). Some nets carry the inverted signal (NOT s2[2], NOT c3[2], NOT c3[3]); this costs nothing because maj3 and xor3 are self-dual (inverting all inputs inverts the output).

4 Part D — Physical design with LibreLane (RTL → GDSII)

`pnr/config.json` sets DESIGN_NAME, VERILOG_FILES (dir::../rtl/…), CLOCK_PORT clk, FP_SIZING absolute and DIE_AREA [0, 0, 150, 150]. The first CLOCK_PERIOD was 1.2 × T_{min} = 1.2 × 2.79 = 3.348 → **3.5 ns**. LibreLane signs off setup timing in 9 corners (tt/ss/ff × nom/min/max parasitics); every failing run failed only because of setup violations in the slow ss_100C_1v60 corner, with DRC and LVS clean. CLOCK_PERIOD was raised by 0.5 ns per run, as required, until the run passed:

Table 4. LibreLane runs (CLOCK_PERIOD +0.5 ns per run).

Run	Clock period (ns)	Setup slack, all corners (ns)	Setup slack, tt (ns)	DRC Magic / KLayout	LVS errors	Result
1	3.5	-2.188	+0.509	0 / 0	0	setup violation
2	4.0	-1.688	+1.009	0 / 0	0	setup violation
3	4.5	-1.188	+1.509	0 / 0	0	setup violation
4	5.0	-0.686	+2.009	0 / 0	0	setup violation
5	5.5	-0.208	+2.509	0 / 0	0	setup violation
6	6.0	-0.154	+2.877	0 / 0	0	setup violation
7	6.5	-0.220	+3.119	0 / 0	0	setup violation
8	7.0	-0.134	+3.520	0 / 0	0	setup violation
9	7.5	-0.104	+3.737	0 / 0	0	setup violation
10	8.0	+0.396	+4.237	0 / 0	0	passed

Table 5. Part D results (final run, CLOCK_PERIOD 8.0 ns).

Metric	Value
CLOCK_PERIOD used (ns)	8.0
Number of runs needed	10 (3.5 → 8.0 ns in 0.5 ns steps)
Core area (µm²)	17759.5 (die area 22500)
Cells after synthesis (from Part B)	61
Cells after place and route	1883 (of which 331 standard cells excluding decap/fill, incl. 245 taps)
Kinds of cells added	tap cells tapvpwrvgnd_1 × 245; decap cells decap_3/4/6/8/12 × 1298; fill cells fill_1/fill_2 × 254; timing-repair buffers buf_2 × 17 and clkdlybuf4s25/4s50 × 4; clock-tree buffers clkbuf_16 × 3
Wirelength (µm)	2499
Setup slack (ns)	0.396 (worst over all corners, ss_100C_1v60; tt corner 4.237)
Post-route F_{max} (MHz)	1000 / (8.0 – 0.396) = 131.51
F_{max} from Part C (MHz)	358.42
DRC errors	0 (Magic), 0 (KLayout), 0 (detailed router)
LVS errors	0 (Netgen)

F_{max} comparison. Post-route F_{max} = 131.5 MHz is 2.7× lower than the 358.4 MHz of Part C. Evaluated in the same typical corner as Part C the post-route value is 1000 / (8.0 – 4.237) = 265.8 MHz, still lower than Part C because of wire RC, the clock tree and the inserted buffers (see Q4).

Cell count comparison. Synthesis produced 61 cells; the final layout has 1883 instances. Most of the difference (1797) is physical-only cells without logic function — 245 well taps and 1552 decap/fill cells that fill every empty site of the rows (utilization is only 7.1 % of the core area inside the 150 µm × 150 µm die). The flow also added 21

timing-repair buffers and 3 clock buffers. (LibreLane runs its own synthesis, giving 16 flip-flops, 42 logic cells and 4 inverters.) Counts are from the final netlist and the design__instance__count__class:* entries of metrics.json.

Remaining warning (not a pass criterion). metrics.json reports 11 max-slew violations, all in the slow ss_100C_1v60 corner (0 in the tt and ff corners; 0 max-fanout and 0 max-capacitance violations). All of them are on the asynchronous reset net: the buffer fanout20 that LibreLane inserted on rst_n drives 10 RESET_B pins with a 0.821 ns transition against the 0.75 ns limit (0.071 ns over). This affects only how sharp the reset edge is, not the data paths; the reset recovery and removal checks still pass with margin in the ss corner (+5.15 ns and +1.30 ns), and the run meets the PDF's criteria (0 DRC, 0 LVS, setup slack ≥ 0).

Layout screenshots (KLayout, final GDS)

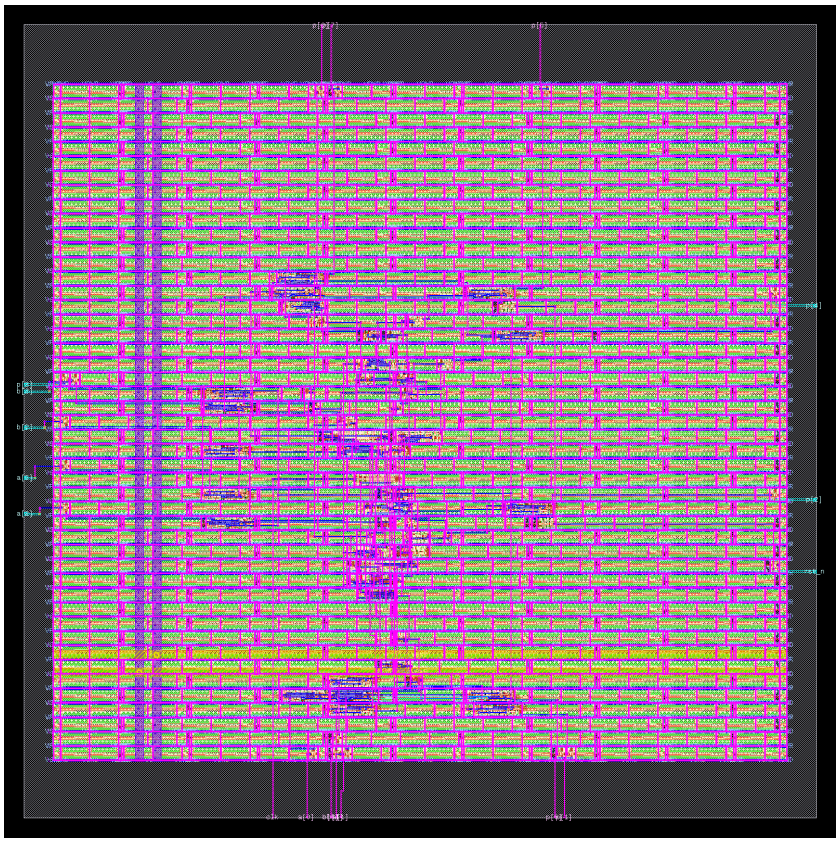


Figure 4. *Whole die (150 µm × 150 µm): I/O pins on the boundary, the power grid (magenta straps and rails), and the standard-cell rows filled with logic, tap, decap and fill cells.*

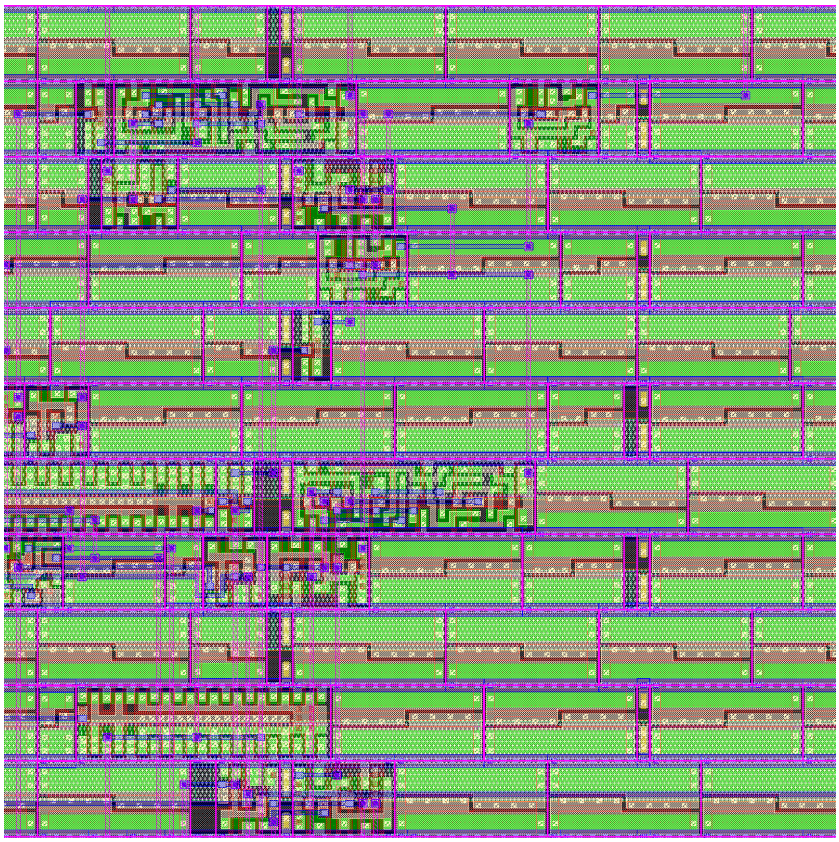


Figure 5. *Zoomed view (30 µm × 30 µm at the centre of the logic): standard-cell rows bounded by the met1 power rails, logic cells between decap/fill cells, and the routed metal wires (met1/met2) and vias.*

5 Additional verification

Table 6. Verification checks.

Check	Result
Exhaustive RTL simulation (256 vectors) and reset check	PASS: 256 vectors checked, 0 errors
Error injection, task 6 (sum = a ^ b)	FAIL: 256 vectors checked, 95 errors
Error injection in the reset value of p (p reset to 5 instead of 0)	FAIL: 256 vectors checked, 1 errors — first line: MISMATCH at 20 ns: p=5 during reset, expected 0
Gate-level simulation of the synthesized netlist	PASS: 256 vectors checked, 0 errors
Independent check of the netlist: all 61 cells evaluated with the official SKY130 functional models (sky130_fd_sc_hd__*.functional.v), all 256 inputs	256 vectors, 0 errors; 16 flip-flops on clk / rst_n with D→Q exactly as in the RTL
Array equations of Section 2 in a bit-level model	256 / 256 products correct; 9 × 7 trace as in Part A
Reproducibility: complete re-run of the committed repository	netlist, stat report and timing report byte-identical; LibreLane at 8.0 ns passes in one run with identical metrics (slack +0.396 ns, 1883 instances, 2499 µm, 0 DRC, 0 LVS)
Committed GDS vs. re-run GDS (geometric XOR in KLayout)	0 differences on all 40 layers, same 33 cells, die 150 × 150 µm

6 Answers to the questions (Section 10 of the assignment)

1. Logic cells after synthesis.

Synthesis produced 61 cells, of which 16 are flip-flops, so **45 cells implement the logic** (instead of 16 + 12 = 28). The number is not simply 28 because the AND gates and full adders are not mapped one-to-one. After flattening, Yosys/ABC optimizes the whole multi-level logic network and maps it onto whatever SKY130 cells are cheapest: a full adder needs a carry cell and a sum cell (maj3 + xor3/xnor3), or several 2-input gates (e.g. xnor2/xor2, nand2, a21oi). ABC does not use the multi-output sky130 fa cell, so each FA becomes at least two cells. On the other hand, adders with constant inputs (r0[3] = 0, c_i[0] = 0) collapse into half adders or simple gates, and many AND gates are absorbed into complex gates (a21oi, a22oi, o311ai, o2111ai, nand3/nand4). Only 4 maj3 and 4 three-input XOR/XNOR cells remain as complete full-adder carry/sum cells, and the rest of the logic is spread over 37 smaller gates.

2. Critical path.

The critical path starts at the input register **a_r[1]** (flip-flop _91_, clock→Q) and ends at the D pin of the output register **p[6]** (flip-flop _88_), i.e. from the a-input register to the product register. Between the two flip-flops it contains **8 combinational cells** (10 instances including both flip-flops): clkinv_1 → o311ai_0 → a21oi_1 → maj3_1 → xnor3_1 → maj3_1 → maj3_1 → xor3_1, with a data arrival time of 2.652 ns. Mapped back to the RTL (by simulating the netlist with the SKY130 cell models and matching each net), the path runs through FA(1,0) carry → FA(1,1) sum → FA(2,0) carry → FA(2,1) carry → FA(2,2) sum → FA(3,1) carry → FA(3,2) carry → FA(3,3) sum, i.e. **8 full-adder stages = 2N**, exactly the estimate: down through the rows and then along the ripple-carry chain of the last row. The number of cells equals the number of adder stages because synthesis implements most stages with one complex cell (maj3 for a carry, xor3/xnor3 for a sum) and merges the two half adders of the first column (constant-0 carry inputs) together with the AND gates into three cells.

3. Flip-flop share of the area.

The 16 flip-flops take 400.384 µm² of 756.976 µm², i.e. **52.89 %**. A dfrtp_1 (flip-flop with asynchronous reset) is a large cell: 25.024 µm², about 3.2× the average logic cell (7.92 µm²). The register count grows only linearly with the width (4N = 16 bits: a, b and the 2N-bit product), whereas the logic grows as N² (16 AND + 12 FA). For N = 4 the array is so small that the fixed register overhead dominates; for wider multipliers the N² logic would take over.

4. Why post-route F_{max} is lower; added cells.

Part C uses ideal conditions: only the typical corner, no wire RC, an ideal clock and the bare synthesized netlist. After place and route LibreLane signs off on the real layout: (a) in the slow corner ss_100C_1v60 (1.60 V, 100 °C), where each cell is roughly 2–2.5× slower (e.g. maj3_1: 0.92 ns instead of 0.40 ns) — this corner sets the worst slack 0.396 ns; (b) with extracted wire resistance and capacitance, so every net adds delay and slows the transitions; (c) with a real clock tree (clock latency and skew) and a 0.25 ns clock uncertainty; and (d) with the buffers the repair steps inserted (on the critical path two clkdlybuf4s25_1 buffers alone add about 1.2 ns in the ss corner). This gives 131.5 MHz instead of 358.4 MHz (265.8 MHz even in the typical corner). Cells added: **tap cells** (245 × tapvpwrvgnd_1) connect the n-well and p-substrate to VPWR/VGND at regular intervals to prevent latch-up; **decap cells** (1298) are on-chip decoupling capacitors between VPWR and VGND that supply current spikes locally and reduce supply noise/IR drop; **fill cells** (254 × fill_1/fill_2) fill the remaining gaps so that wells, implants and power rails are continuous along each row and density rules are met; **buffers** — 17 × buf_2 and 4 × clkdlybuf4s* timing-repair buffers fix maximum slew/capacitance/fanout on long or heavily loaded nets (e.g. the reset net and input ports driving several gates), and 3 × clkbuf_16 form the clock tree so the clock reaches all 16 flip-flops with strong edges and almost no skew.

5. DRC and LVS.

DRC (design rule check, done by Magic and KLayout) verifies that the layout geometry obeys the foundry's manufacturing rules for every layer: minimum width and spacing, enclosure and overlap of vias and contacts, minimum area, density and antenna rules. **LVS** (layout versus schematic, done by Netgen) extracts the transistors/cells and their connectivity from the layout and checks that this circuit is identical to the gate-level netlist — same devices, same nets, no shorts, no opens, matching pins. Both must pass: a DRC violation means the masks cannot be printed reliably (shorts, opens, low yield), so the chip may not work or the foundry will not accept it; an LVS mismatch means the manufactured chip would be a different circuit from the one that was simulated and timed, so all verification would be invalid. This design has 0 DRC errors and 0 LVS errors.

6. Scaling to 8 bits.

Using the formulas of Section 2: full adders $N(N-1)$: $4 \cdot 3 =$ **12** $\rightarrow$ $8 \cdot 7 =$ **56** (4.7× more); AND gates N^2: 16 $\rightarrow$ 64; flip-flops $4N$: 16 $\rightarrow$ 32. Critical path about $2N$ full-adder delays: $2 \cdot 4 = 8 \rightarrow 2 \cdot 8 =$ **16**, i.e. the combinational delay roughly doubles (grows linearly with N). With our numbers the 4-bit path is 2.652 ns including 0.398 ns clock→Q, i.e. about 2.25 ns of logic for its 8 full-adder stages (≈ 0.28 ns per stage); 16 stages give roughly 0.40 + 2 × 2.25 + 0.14 (setup) ≈ 5.0 ns, so $T_{min} \approx 5.0$ ns and $F_{max} \approx 198$ MHz (about half of 358 MHz), while the adder area grows about 4.7×. The latency stays 2 cycles but each cycle is about twice as long.